Chapter 15

Probability functions

by Lorraine Gaudio

Version June 22, 2026

Contents

1 Overview

Probability is easier to understand when you can build and inspect examples yourself. In this chapter, you will use R to simulate random outcomes, visualize those results, and decide whether the patterns you see are reasonable. You will work with coin flips, dice rolls, bell-shaped data, and repeated simulations so that probability feels concrete rather than abstract. These are the same kinds of tools analysts use when they need to study uncertainty, test assumptions, or estimate risk.

By the end of Chapter 15 you will be able to:

1. Remember – Name `sample()`, `rnorm()`, and `replicate()`.
2. Understand – Explain how replacement and probability weights affect simulation results.
3. Apply – Simulate coins, dice, and normally distributed data with R code
4. Check – Use tables, proportions, and histograms to verify whether simulated results make sense.
5. Interpret – Describe sampling variability and distinguish between fair and biased outcomes using simulated evidence.

Keep these goals in mind as you move through each section.

2 Setting Up Your Notes

To begin Chapter 15's guided activity, follow these steps:

1. Open your course project for RStudio

For those of you who **have GitHub** and a course project, navigate to your course folder and open your course project. This will open RStudio. Click "Pull" in the Git pane to get the latest version of the course materials. After completing work, save, stage, commit, and push.

If you **don't have GitHub** or a course project, just open RStudio and set your working directory to your course folder.

2. Create a new file

Create a new Quarto document for Chapter 15 and save it with a chapter-specific file name like `chapter15_notes.qmd`

3. Follow along

- Type in the goal, code, and comments provided in this document. Use memos to explain your thinking. Typing out journal-like reflections will help you internalize the concepts and make the learning more active.
- Keep your work organized with headings. Since this chapter uses random simulation, run your document from top to bottom when you want reproducible results.

2.1 Packages

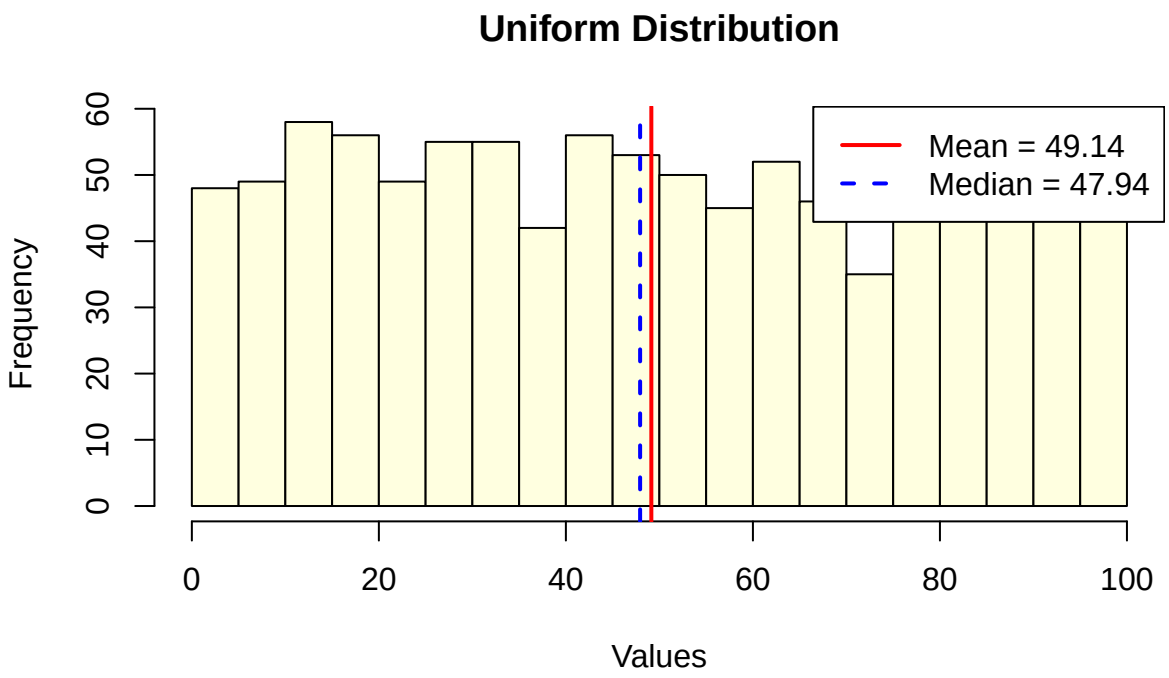
No extra packages are needed today. We'll be using built-in stats functions.

3 Distribution of Sums

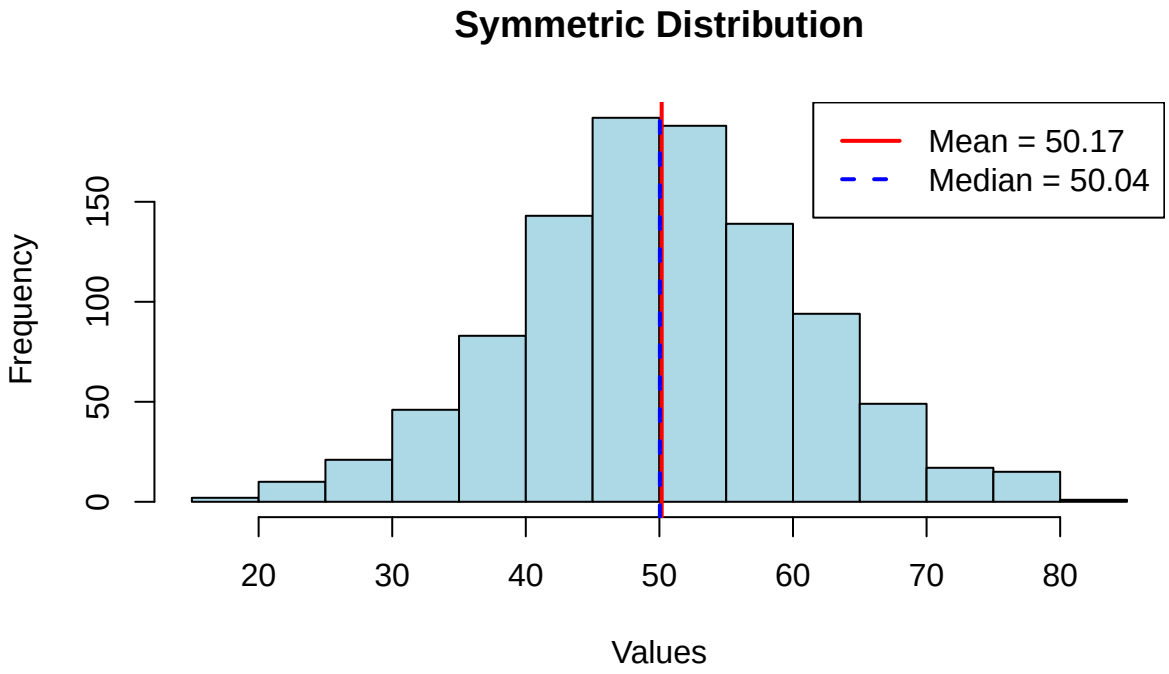
Identify the "Shape of Data" (or "Distribution of data") is *optional reading*, but may help you answer questions in this chapter.

Below are examples of different histogram shapes that you may encounter in real data. **Histograms** are the count of values in each bin (or bar) of a histogram. The shape of the histogram can provide insights into the underlying **distribution** of the data, such as whether it is uniform, symmetric, skewed, or bimodal.

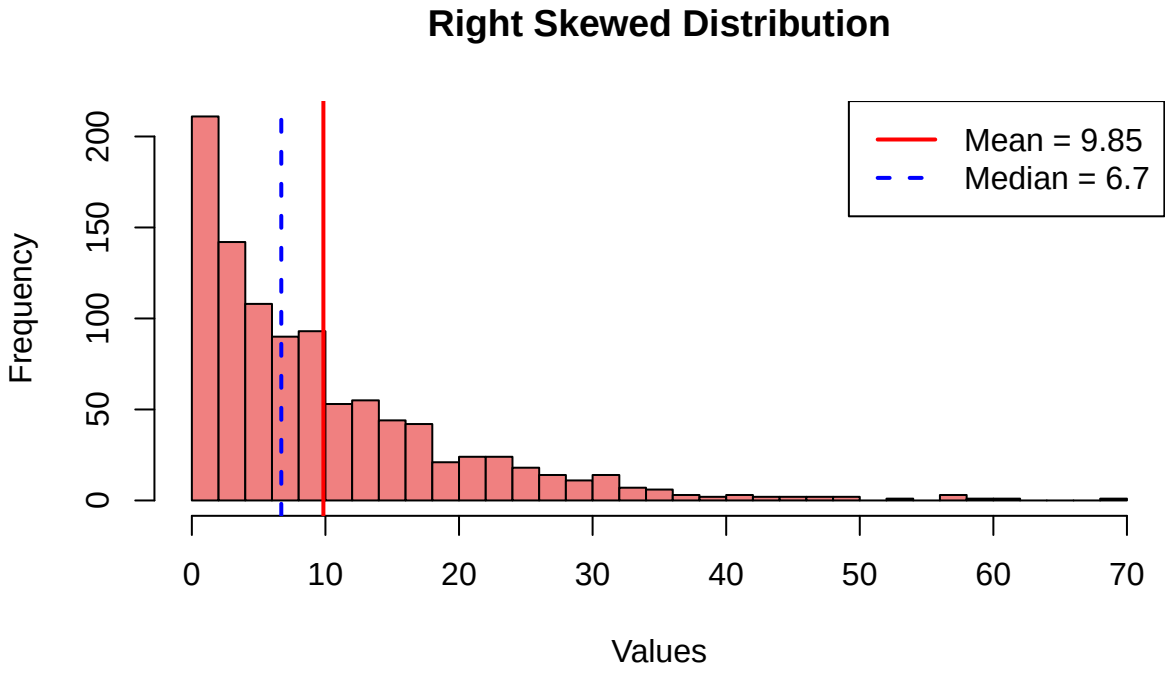
1. **Uniform**: All bars have approximately the same height, indicating that all values are equally likely.



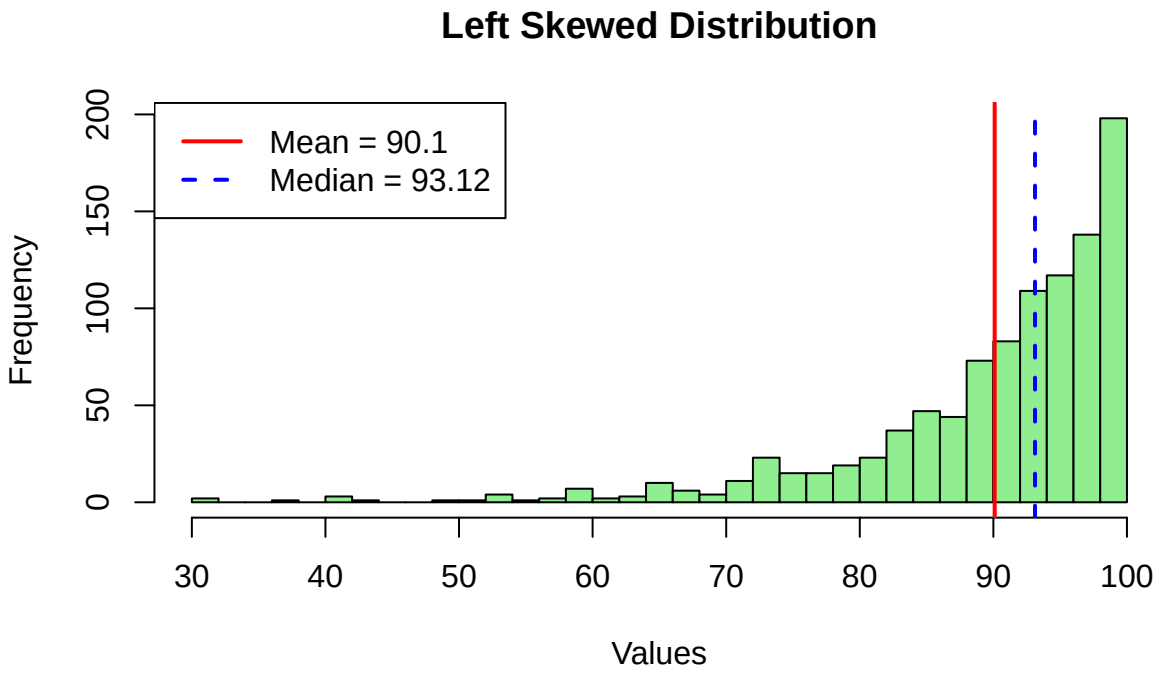
2. **Normal**: This bell-shaped (symmetric) distribution is the left and right sides of the histogram are nearly mirror images of each other. Mean and median are approximately equal. $Median \approx Mean$.



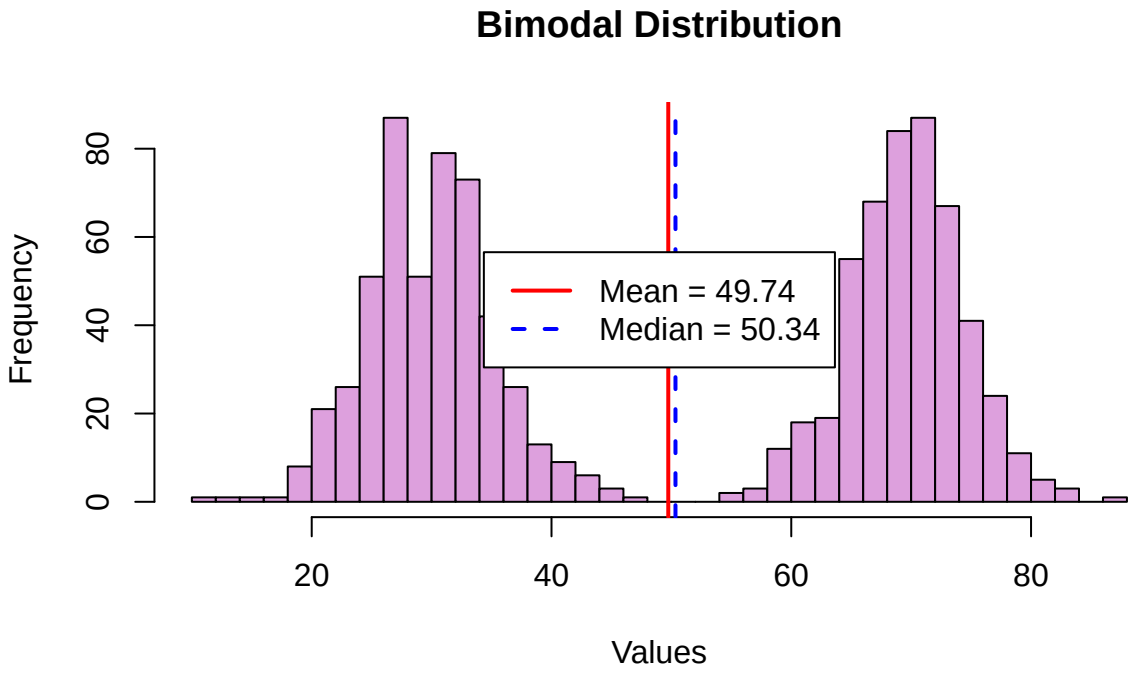
3. **Skewed Right**: The right *tail* is longer than the left tail, indicating that there are a few high values that pull the mean to the right. *Positive skewness.* The mean is more right than the median due to extreme values. $Median < Mean$.



4. **Skewed Left**: The left *tail* is longer than the right tail, indicating that there are a few low values that pull the mean to the left. *Negative skewness.* $Median > Mean$.



5. Bimodal (Split Distribution): The distribution has two distinct peaks or modes. This suggests that the data might come from two different groups or processes. The mean and median may fall between the peaks, providing misleading summary statistics. Bimodal distributions often indicate hidden sub-populations.



4 Warm-Up

A data analyst uses `set.seed()` to ensure that random processes like sampling or simulation produce the same results each time the code is run (from top to bottom). This **reproducibility** is crucial for debugging, collaboration, and verifying results, as it guarantees consistency across analyses and users.

```
#  Test
set.seed(2025)  #  for reproducibility
```

If you re-run code more than once within your document, your values will not be the same. It is only the same when you run the code from top to bottom in one session.

5 Simulating

Probability is the study of random outcomes. To understand probability, we need to be able to **simulate** random outcomes and inspect those results. R provides powerful tools for simulating random data, which allows us to explore concepts like **sampling variability**, distributions, and the effects of bias.

5.1 Coin Flips

In this first example, we'll simulate flipping a coin 1,000 times and counting how many times it lands on tails. This simple experiment helps us understand the concept of a sample space, the role of randomness, and how to verify our results.

Flip 1,000 fair coins (0 = heads, 1 = tails) and count tails.

Sample Space "S" is a collection (or set) of all possible outcomes in a random experiment.

Set Notation: $S = \{\text{Head, Tail}\}$

1. Create the data through a simulation

```r
#  Test
# sample(x, size, replace = FALSE, prob = NULL)
coins <- sample(x = 0:1, size = 1000, replace = TRUE)
```

`x = 0:1` means that we are sampling from either 0 or 1. As the researcher, I decide that 0 = heads and 1 = tails. We set `size = 1000` to flip the coin 1,000 times. We set `replace = TRUE` because we "put the coin back" each flip.

replacement means that after we sample a value, we *"put it back"* into the pool of possible values for the next draw. This allows for the same value to be drawn multiple times, which is essential for simulating **independent events** like coin flips.

```r
#   Verify
unique(coins)   # values in our data set
length(coins) # total number of flips
typeof(coins)  # type of object
```

Coins contains 0 and 1 values where 1 represents tails of a coin flip. There are 1000 values in the integer vector.

Prediction: How many tails are in our data set?

Use sum to count the number of tails because 1 = tails

```r
#  Test
sum(coins)
```

Comment (your turn): How many tails are in our data set? Why does `sum(coins)` count the tails?

Explore and Play: Can you make a new data set where heads = 1? Try `1 - coins` and sum. Why would this work?

Break Things! What happens if `replace = FALSE` keeping `size = 1000`? Memo: paste the error message and 1 sentence on why it occurs.

5.2 Dice Rolls

Simulate 10,000 rolls of a fair 6-sided die and plot the results.

Set Notation: $S = \{1, 2, 3, 4, 5, 6\}$

Prediction: How many times do you expect each face to appear? Will the values be exactly equal? Have a normal distribution? Something else? Type your answer before working ahead. *Keep whatever you typed, even if you are wrong. That's the integrity of science!*

```r
#  Test
dice <- sample(1:6, 10000, replace = TRUE)
```

To investigate our hypothesis, we'll visualize the results with a histogram using base R.

6 Histograms

Use `hist()` to create a histogram of the dice rolls, showing the frequency of each face value.

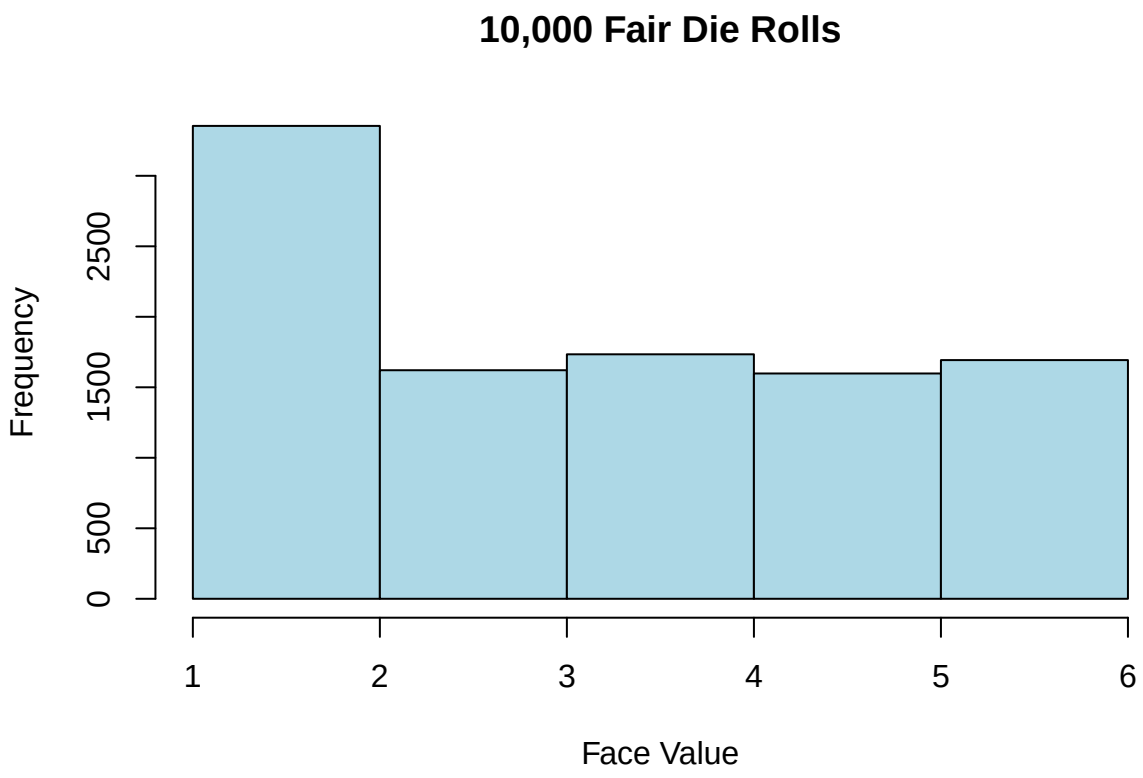
```r
?hist
```

The SYNTAX: `hist(x, breaks, col, main, xlab, ylab)`

- `x` is the data to plot
- `breaks` controls the number of bins (or bars)

- `col` sets the color of the bars
- `main` is the title of the plot
- `xlab` and `ylab` label the x- and y-axes

```
#   Verify
hist(dice,                  # The data to plot
     breaks = 6,
     col    = "lightblue",  # Color of the bars
     main   = "10,000 Fair Die Rolls", # Title of the plot
     xlab   = "Face Value") # Label for the x-axis
```



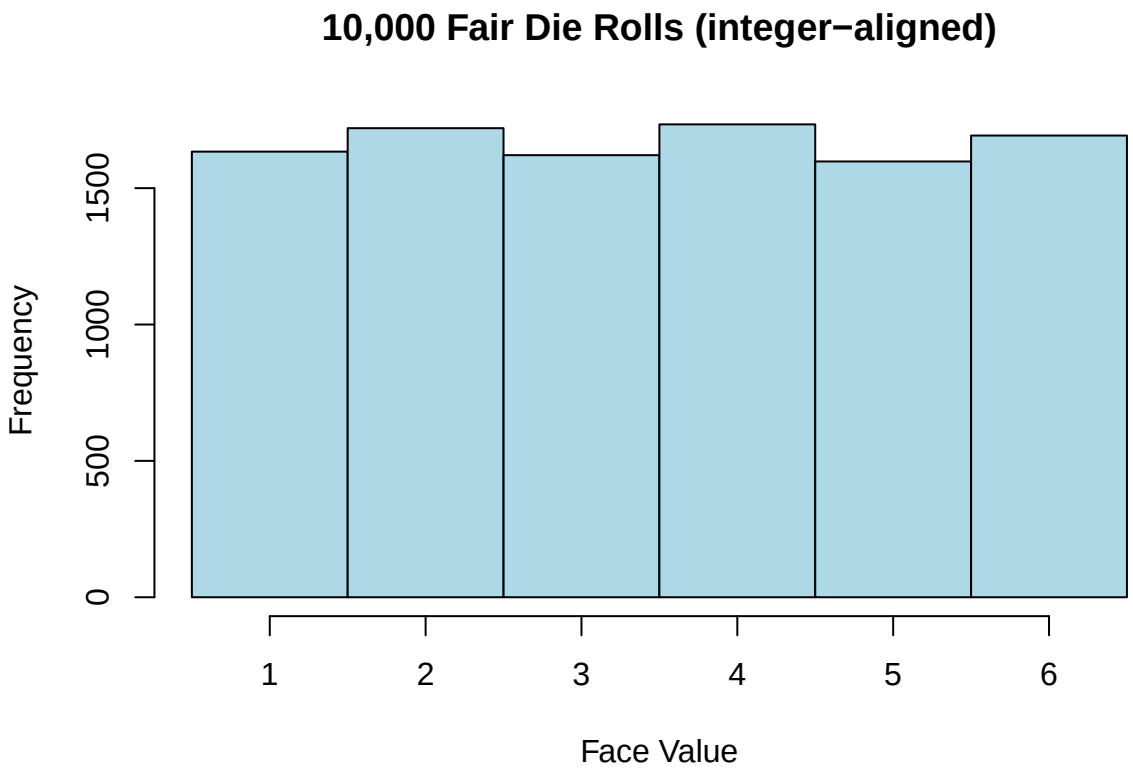
The y-axis label is automatically set to "Frequency" by default.

NOTICE: Bars should be roughly equal height, forming a uniform distribution. That isn't what we see.

When we specify breaks = 6, base R doesn't create bins that align perfectly with our integer values 1-6. The bins slice the integers unevenly, leading to misleading bar heights.

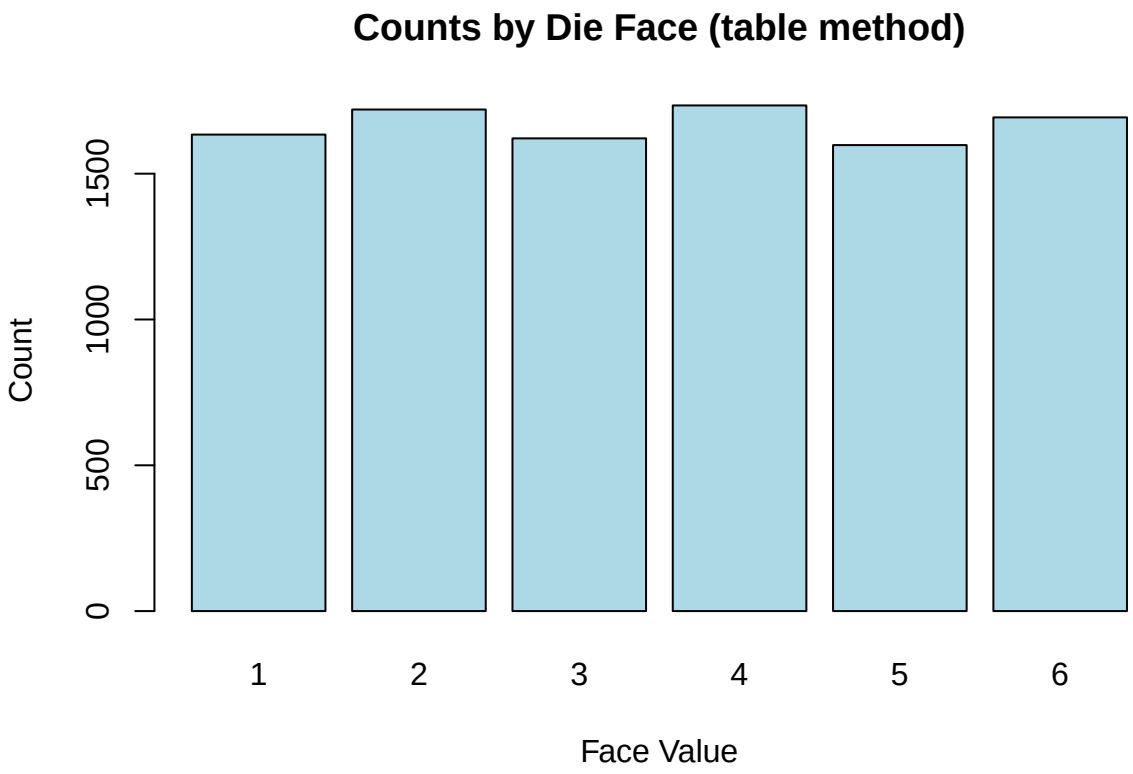
Fix the breaks to align with die face values.

```
#  Test
hist(dice,
     breaks = seq(0.5, 6.5, by = 1),  # centers bars on 1:6
     col    = "lightblue",
     main   = "10,000 Fair Die Rolls (integer-aligned)",
     xlab   = "Face Value")
```



The bars look more even now.

```
# Alternative display for discrete data
barplot(table(dice),
        main = "Counts by Die Face (table method)",
        col    = "lightblue",
        xlab = "Face Value",
        ylab = "Count")
```



The table() function provides an alternative way to display the data accurately.

7 Two Dice

Prediction: What histogram shape do you expect when summing the face value of two dice? Will the values be exactly equal? Have a normal distribution? Something else? Type your answer before working ahead. *Keep whatever you typed, even if you are wrong.*

Roll two dice 10,000 times, add the results, plot.

```
#  Test
sum_two <- sample(1:6, 10000, TRUE) + sample(1:6, 10000, TRUE)

#   Investigate
hist(sum_two,
     breaks = seq(1.5, 12.5, by = 1),  # Creates bins centered on integers 2-12
     col    = "salmon",
     main   = "Sum of Two Dice (10,000 trials)",
     xlab   = "Total")
```

The histogram shows a bell-shaped (normal) distribution, peaking around 7. This shape arises because there are more combinations to achieve sums near the middle (like 6, 7, and 8) compared to extreme sums (like 2 or 12).

Explore and Play: We used the column "salmon" to create the figure. Test out other `col` options. Which are your favorite?

```r
colors()  # Lists all color names in R
```

---

8 Symmetric Data

The functions `rnorm()` generates a vector of random numbers that form a normal (bell-shaped) distribution.

```r
?rnorm  # Check the help file for rnorm()
```

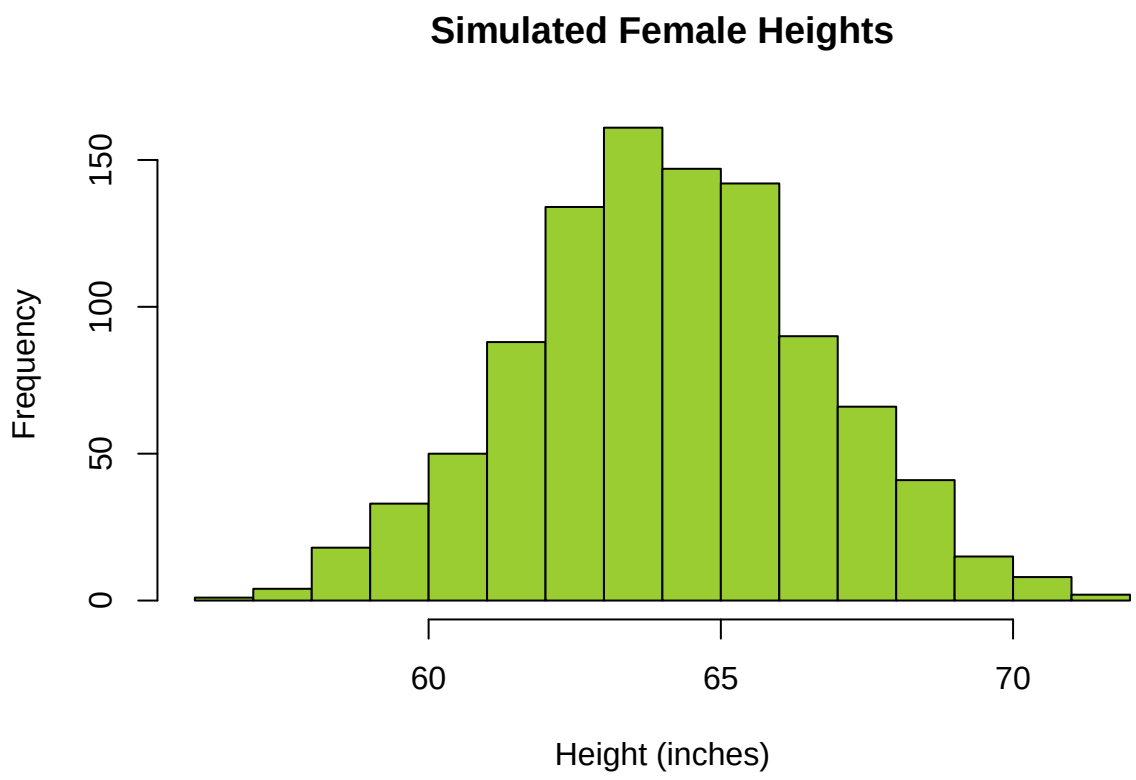
The SYNTAX: `rnorm(n, mean = 0, sd = 1)`

- `n` is the number of random numbers to generate.
- `mean` is the average value of the distribution.
- `sd` is the standard deviation, controlling the spread of the distribution.

Simulate 1,000 adult female heights $\sim N(64, 2.5^2)$ and inspect.

```r
#  Test
heights <- rnorm(n = 1000, mean = 64, sd = 2.5)
```

```r
#   Verify
hist(heights,
     col    = "yellowgreen",
     main   = "Simulated Female Heights",
     xlab   = "Height (inches)")
```



The `breaks =` are unnecessary here because `hist()` automatically chooses appropriate bins for continuous data.

```
#   Verify
"Mean = "
```

```
## [1] "Mean = "
```

```
mean(heights)
```

```
## [1] 64.11653
```

```
"SD = "
```

```
## [1] "SD = "
```

```
sd(heights)
```

```
## [1] 2.508912
```

The histogram shows a bell-shaped distribution of heights, centered around the mean of 64 inches. The spread of the data is consistent with the specified standard deviation of 2.5 inches.

Sample vs. Population Parameters: You might wonder, "Why aren't the sample mean and standard deviation exactly 64 and 2.5?" This is because these are random samples from a normal distribution with those parameters. Each time we run the simulation, we get slightly different values due to **sampling variability**.

In summary, the function `rnorm()` generates random numbers from a normal distribution and is used for modeling normally distributed data using the parameters: number of values, mean, and standard deviation. This will only generate symmetric data.

9 Repeating Simulation

The function `replicate()` repeats an expression multiple times and collects the results. This control structure is like a loop is used to repeat sample estimate probabilities (simulations), resample data to estimate variability or confidence intervals (bootstrapping), model repeated trials of an experiment, generate synthetic data and more. This function is flexible as it can wrap any code, not just random generation.

That said, **Monte Carlo simulations** are the classic use case for `replicate()`. These simulations model are used to understand and manage risk and uncertainty using probabilities.

```
?replicate  # Check the help file for replicate()
```

The SYNTAX: `replicate(n, expr, simplify = "array")`

- `n` is the number of replications.
- `expr` is the expression (a language object, usually a call) to evaluate repeatedly.
- `simplify` indicates whether the result should be simplified to a vector, matrix or array if possible. The default is `"array"`.

How variable is the number of tails in 5,000 flips? Repeat 100 times.

…. That's a lot of coins tosses!

```
#  Test
tails_100 <- replicate(n = 100, expr = sum(sample(0:1, 5000, TRUE)))
```

The `replicate()` function:

1. Evaluates the entire expression 100 times (because `n = 100`)
2. Each evaluation is independent (gets its own random sampling)

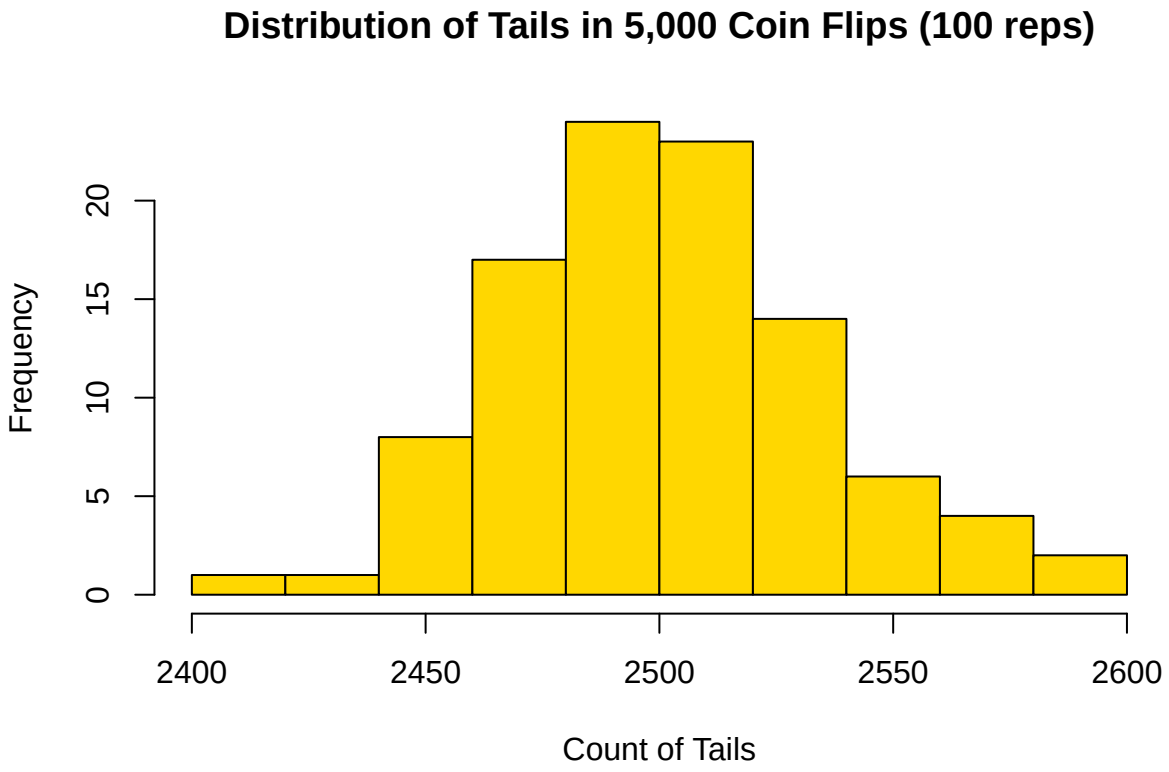
3. Store the 100 results in a vector called `tails_100`

```
#   Verify
summary(tails_100)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    2412    2477    2499    2502    2525    2582
```

Sampling variability show that the number of tails varies across these simulations, with a minimum of 2,412 tails and a maximum of 2,582 tails. The mean number of tails is approximately 2,499, which is close to the expected value of 2,500 for a fair coin.

```
#   Verify
hist(tails_100,
     col    = "gold",
     main   = "Distribution of Tails in 5,000 Coin Flips (100 reps)",
     xlab   = "Count of Tails")
```



The spread shows sampling variability around the theoretical 2,500.

9.1 Normal Data with `replicate()`

Scenario. Pretend that the university is considering making specifications grading standard practice. The provost's office says that if thirty percent or more of students oppose this, it would trigger a halt to the proposal. We want to know the risk of the proposal being halted.

We run a survey where student attitudes are measured on a continuous scale:

- 0 = neutral,

- negative values = oppose,
- positive values = support.

We'll first run a random-sample survey of 50 students. Then we simulate many 50-student samples to see how much the number of opponents could vary. That variability informs the proposal planning.

9.1.1 Opinion Study

Generate 50 random numbers, compare each to zero, and count how many are below zero.

```
set.seed(1234)  # For reproducibility
# Generate 50 random numbers to represent student attitudes from survey
# with a normal distribution (mean = 2, sd = 3)
x <- rnorm(n = 50, mean = 2, sd = 3)
print(x)
```

While the average opinion is supportive (mean = 2), some individuals express opposition (negative values). In other words, some values are below zero because the mean is 2, but the standard deviation is 3, allowing for negative values. That's normal sample-to-sample variation.

```
# Mark who opposes (score < 0), then count
below_zero <- x < 0
print(below_zero)

# Count how many are below zero
count <- sum(below_zero)
print(count)
```

We generated 50 random numbers from a normal distribution with mean 2 and standard deviation 3. We then compared each number to zero, yielding a logical vector indicating which are below zero. Finally, we summed the logical vector to count the number of values below zero.

9.1.2 Many Samples Simulation

Now simulate many realistic samples to understand how the count of opponents might bounce around.

Repeat this process multiple times using `replicate()`. This time, let's generate 50 random numbers in 200 iterations.

Use `replicate` to repeat the counts across many simulated samples.

```
#  Test
values_below_zero <- replicate(
  n = 200,                                    # 200 hypothetical survey runs
  expr = sum(rnorm(50, mean = 2, sd = 3) < 0)) # each run: 50 students, count opponents
```

Order of Operations:

1. `rnorm(n = 50, mean = 2, sd = 3)` generates a sample of 50 random numbers.
2. `sum(... < 0)` counts how many of those numbers are below zero.
3. `replicate(n = 200, expr = ...)` repeats this process 200 times.
4. The result is a vector of counts, one for each of the 200 samples.

```
#   Investigate
# Useful summaries for discussion
mean(values_below_zero)   # typical number of opponents
```

```
## [1] 12.77
```

```r
sd(values_below_zero)     # how much the count varies
```

```
## [1] 3.121831
```

```r
quantile(values_below_zero, c(.1,.5,.9))  # 10th/50th/90th percentiles
```

```
## 10% 50% 90%
##   9  13  17
```

The mean indicates that, on average, the typical count of those opposed is about 13 across many samples of 50 students. The standard deviation shows that the count of opponents can vary by about 3.1 from sample to sample. The percentiles provide additional insights into the distribution of opponent counts, indicating that in 10% of the samples, there are 9 or fewer opponents, while in 90% of the samples, there are 17 or fewer opponents.

```r
# Example planning thresholds (adjust to your context)
mean(values_below_zero >= 15)  # How often do we see  15 opponents out of 50?
```

```
## [1] 0.28
```

If at least 15 opponents trigger a halt, the proposal's risk of being halted is about 28% based on our simulations.

```r
# Alternative way to calculate probability of halt
# One-line simulation estimate:
risk_halt <- mean(replicate(200, sum(rnorm(50, 2, 3) < 0)) >= 15)
risk_halt
```

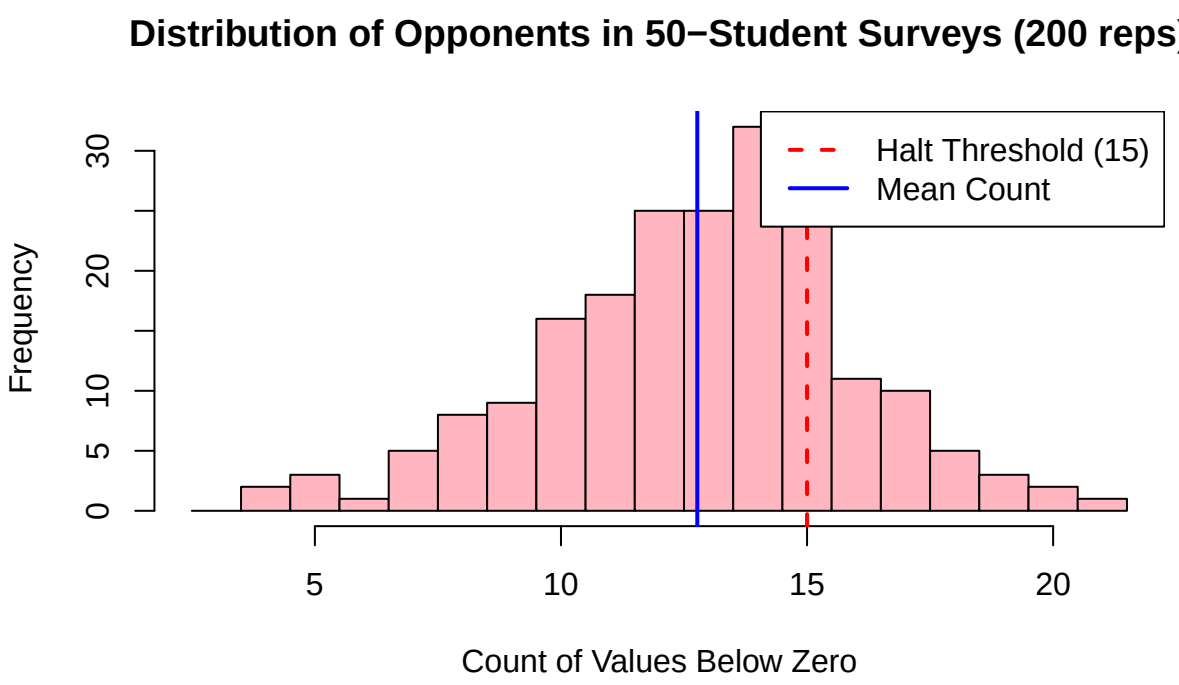
```
## [1] 0.2
```

```r
# Integer-aligned histogram + shading + on-plot probability
threshold <- 15
max_count <- max(values_below_zero)
breaks_int <- seq(2.5, max_count + 0.5, by = 1)

#   Investigate
hist(values_below_zero,
     breaks = breaks_int,
     col    = "lightpink",
     main   = "Distribution of Opponents in 50-Student Surveys (200 reps)",
     xlab   = "Count of Values Below Zero")


abline(v = threshold, col = "red", lwd = 2, lty = 2) # Halt threshold
abline(v = mean(values_below_zero), col = "blue", lwd = 2, lty = 1)  # Mean line

legend("topright",
       legend = c("Halt Threshold (15)", "Mean Count"),
       col = c("red", "blue"),
       lty = c(2, 1),
       lwd = 2,
       bg = "white",
       bty = "o")
```



Each bar shows how often a particular count of opponents appeared across the 200 samples. The center of the histogram is the typical count. The spread shows how much that count changes from sample to sample, even if the underlying social climate doesn't change. About 1 in 4 samples hit the halt threshold (15).

10 Fair-Dice Check

Probability weights in `sample()` let you simulate biased outcomes. Recall the syntax:

The SYNTAX: `sample(x, size, replace = FALSE, prob = NULL)`

A **fair outcome** means that each outcome in the sample space has an equal chance of occurring. For example, a fair six-sided die has an equal probability of landing on any of its faces (1 through 6), which is 1/6 or approximately 0.1667 for each face.

10.1 Biased Die

A **biased outcome** means that some outcomes are more likely to occur than others. For example, a biased six-sided die might have a higher probability of landing on certain faces, such as 1 and 6, while other faces (like 2, 3, 4, and 5) have lower probabilities.

The `prob` parameter specifies the probability weights for each element in the sampling pool.

Simulate rolling a biased die 1,000 times with specified probabilities.

```
#  Test
rolls <- sample(1:6, 1000, prob = c(.12,.13,.07,.23,.22,.23), replace = TRUE)
```

This simulates rolling one biased or "loaded" die `1000` times. The length of the prob vector must match the number of sampled values (x = `1:6`). The weights usually sum to 1; R will normalize them if they don't.

```
#   Verify that relative frequencies should be near the weights
round(prop.table(table(rolls)), 3)
```

```
## rolls
##     1     2     3     4     5     6
## 0.130 0.133 0.076 0.222 0.223 0.216
```

The relative frequencies should roughly match the specified probabilities, confirming the bias in the die. The results may vary slightly due to randomness, but over many rolls, they should converge to the specified probabilities.

Break Things! What happens if the length of `prob` does not match the length of `x`? Paste the error message and 1 sentence on why it occurs. What happens if the `prob` values do not sum to 1? Can you have negative probabilities? You could try `sample(1:6, 10, replace = TRUE, prob = c(.5, .5))` and try `sample(1:6, 10, replace = TRUE, prob = c(.1, .2, 0.2, .05, .2, .05, .4))` and `sample(1:6, 1000, prob = c(.22,-.13,.07,.23,.22,.23), replace = TRUE)`. Explain what happens and what the rules are for the `prob` argument in `sample()`. _____

10.2 Fair vs. Unfair Dice

Scenario. You are playing a dice game with a "friend" who claims their dice are fair. Your "friend's" dice look suspiciously unfair. Compare it to our fair dice. We'll test the your friend's dice against your fair dice.

Simulate rolling two biased dice 1,000 times and compare to two fair dice. We'll compare your friend's results to fair dice by adding the the face values of the dice then

```r
#  Test
set.seed(8000)
biased <- c(.12, .13, .07, .23, .22, .23)  # sums to 1
friend_sum <- sample(1:6, size = 1000, replace = TRUE, prob = biased) +
  sample(1:6, size = 1000, replace = TRUE, prob = biased)
```

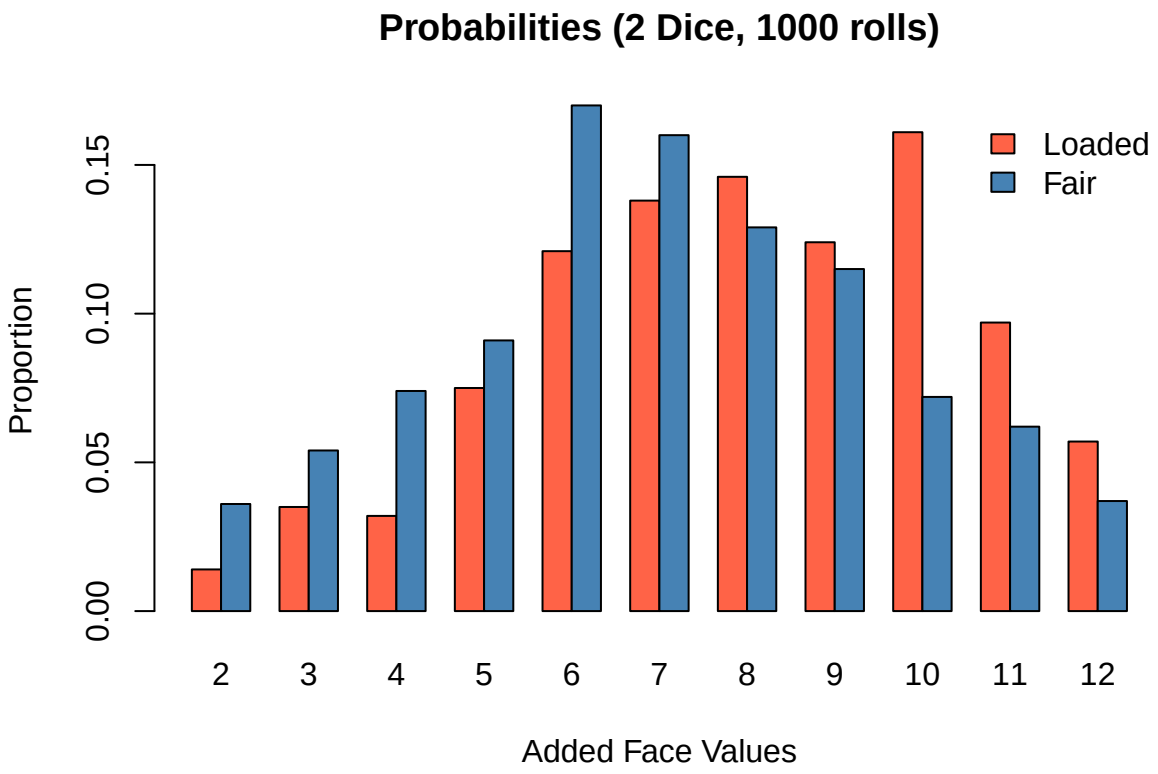
```r
#  Test
fair_sum <- sample(1:6, 1000, TRUE) + sample(1:6, 1000, TRUE) # no 'prob' = equal chance
```

The simulates rolling two biased dice 1000 times. This is like you and a friend each rolling two dice 1000 times simultaneously.

Compare the distributions of the sum of two fair dice face values from your friend's biased dice.

```r
# Proportions of each sum
ploaded <- prop.table(table(friend_sum))
pfair <- prop.table(table(fair_sum))
```

```r
#   Investigate
# Side-by-side bar plot
face_props <- rbind(Bias = ploaded, Fair = pfair)
barplot(face_props,
        beside = TRUE,
        col    = c("tomato", "steelblue"),
        main   = "Probabilities (2 Dice, 1000 rolls)",
        xlab   = "Added Face Values",
        ylab   = "Proportion")
legend("topright", legend = c("Loaded", "Fair"),
       fill = c("tomato", "steelblue"), bty = "n")
```



Comment: What is the shape of the red bars compared with the blue?

11 Summary

In this chapter, you used R to study probability through simulation. You learned how `set.seed()` supports reproducibility, how `sample()` can simulate discrete outcomes such as coin flips and dice rolls, and how `hist()` helps you visualize the shape of simulated results. You also used `rnorm()` to generate bell-shaped data and `replicate()` to repeat simulations so you could examine variability across many runs. Finally, you used probability weights in `sample()` to compare fair and biased outcomes and to judge whether simulated results matched what you would expect from a fair process.

12 Chapter Terms

Biased Outcome: An outcome in a random process where some results are more likely than others, so the outcomes are not equally likely.

Distribution: The pattern of possible values of a variable, together with how likely those values are or how often they occur.

Fair Outcome: An outcome in a random process where all possible results are equally likely. For example, in a fair six-sided die, each face has probability 1/6.

Histogram: A graph for numerical data that groups values into intervals (bins) and shows how many observations fall in each bin.

Independent Events: Events where the occurrence of one does not change the probability of the other. In probability notation, events A and B are independent when $P(A \cap B) = P(A)P(B)$; equivalently, knowing

that one happened does not change the probability of the other.

Monte Carlo Simulation: A simulation method that uses repeated random sampling to approximate probabilities, study uncertainty, or estimate numerical results.

Normal Distribution: A continuous, bell-shaped distribution that is symmetric around its center, with most values near the mean and fewer values farther away.

Probability: A number from 0 to 1 that describes how likely an event is to occur, where 0 means impossible and 1 means certain.

Probability Weights: Numbers that control how likely each value is to be selected in a random sample. In R, the prob argument in `sample()` supplies these weights; they must be nonnegative and do not have to add up to 1.

Replacement: A sampling method where a selected value is put back before the next draw, so it can be chosen again. In R, this is controlled by replace = TRUE in functions like `sample()`.

Reproducibility: The ability to get the same computational results again by using the same data, code, steps, and analysis conditions.

Sample Space: The set of all possible outcomes of a random experiment.

Sampling Variability: The natural sample-to-sample differences that occur because different random samples from the same population usually produce different statistics.

`set.seed()`: An R function that sets the starting state of the random number generator so random operations can be repeated and produce the same results again.

Simulation: A model of a random process that generates artificial outcomes so you can study what might happen and estimate probabilities or patterns.

Uniform Distribution: A distribution in which all possible outcomes in the range are equally likely.

13 Practice Space

In this Practice Space, you are expected to:

- simulate outcomes using functions such as `sample()` and `rnorm()`
- use `replicate()` when a task asks you to repeat a simulation many times
- store requested objects with the exact names given in the task
- check your results using tools such as `head()`, `table()`, `mean()`, `length()`, `any()`, or other appropriate checks
- write short interpretations that explain what the simulation shows, not just what code you ran

When a task asks a yes/no question or asks whether a result seems reasonable, answer in a short memo after your code and checks.

Replace every `____` placeholder and any TODO comment with working code or a short written response. Run each section and make sure the requested objects appear in the Environment. For every task, use the pattern Goal → Code → Checks → Explain.

You will submit your `chapter15_practice.qmd`, your rendered `chapter15_practice.html`(optional), and your `chapter15_practice.RData` workspace file.

13.1 Setup

1. Create a **new Quarto file** in your course folder and save it as `chapter15_practice.qmd`.

- Test Render your document often so you can catch formatting problems and code errors early.

- You may submit the rendered file `chapter15_practice.html` if you successfully create it, but the HTML file is optional.

2. Use the heading structure taught earlier in the chapter.

Your Quarto setup for Chapter 15 should include headings, section numbering, and a table of contents, and you should render often to check both formatting and errors.

3. No extra packages are required

This chapter uses base R functions for probability and simulation, so your code should run cleanly in a fresh R session without loading extra packages.

4. Make your simulation work reproducible.

- Because this chapter uses random simulation, use `set.seed()` when you want your simulated results to be reproducible.
- Run your document from top to bottom to confirm that your output, objects, and explanations all match.

5. Memo your work with the / / /

- a memo before the code chunk that states the goal of the code and what you are trying to learn, check, or demonstrate
- the code chunk itself
- any needed checks after the code
- a memo after the code chunk that explains what you learned, discovered, or confirmed
- answers to all comment questions included in the task

If you **used help** while working, document it in a memo. For example, if you used an LLM or another outside resource to understand an error or concept, briefly disclose that use and explain what help you received. **Type all code yourself.** This is part of the course expectation for AI-supported learning and memo-based workflow.

Follow this template for each coding task.

What is the goal of the code? What am I trying to learn, check, or demonstrate?

```
#Code
your_code <- "here"
```

```
#  Include checks that fit your object
head(your_code)
class(your_code)
length(your_code)
table(your_code)
mean(your_code)
any(your_code)
```

/ What I learned / Next step. 1–2 sentences.

13.2 Task 0

Make your simulation work reproducible.

Set a seed for reproducibility. We'll use the same seed so everyone gets the same simulated results.

```r
#   Set seed for reproducibility
set.seed(2025)
```

Explain how `set.seed()` supports reproducibility in your own words. Why does it need to be near the top of your document before any simulation code?

13.3 Task 1

Scenario. Your friend shows up to your DnD game with a big 3D printed 20-sided die and claims it is fair. You decide to test it. For practice, you will simulate what 300 rolls of a fair 20-sided die can look like, then check whether the results look roughly fair overall. How to Make Fair DnD Dice from ANY Shape

Simulate 300 rolls of a fair 20-sided die using the values `1:20` and store the result in **`big_die`**.

```r
#
big_die <- sample(___, size = ___, replace = ___)

big_die_counts <- table(big_die)

# Did you roll a 20?
round(prop.table(big_die_counts), 3)
range(big_die_counts)
mean(big_die == ___)
```

Comment: Does this simulated die look roughly fair? Explain using the overall pattern of counts, not just whether a 20 appeared. A fair die should have counts that are similar but not exactly equal.

13.4 Task 2

Scenario. A game store is running a weekend probability demo and asks you to help analyze whether a 20-sided die behaves the way a fair die should over many rolls. Your job is to simulate many D20 rolls, check the results, and decide whether the pattern looks reasonable for a fair die.

13.4.1 Step 1

D20 marathon

Simulate 5221 rolls of a fair 20-sided die. Store as **`boring_weekend`** using `sample()`.

```r
#
boring_weekend <- sample(___, size = ___, replace = ___)
```

What do you expect the distribution of rolls to look like? Will the values be exactly equal? Have a normal distribution? Something else? Type your answer before working ahead. *Keep whatever you typed, even if you are wrong. That's the integrity of science!*

13.4.2 Step 2

Quick frequency check

Create **`d20_counts`**: a table of how many times each face appeared.

```r
#
d20_counts <- __(_______)
```

```r
#  Quick check
print(d20_counts)
```

Comment on how similar or different the counts are across the values in the table.

13.4.3 Step 3

Hit rate

Find the proportion of rolls that were natural 20s. Then compare that result to the fair-die benchmark of 0.05.

```
#
mean(boring_weekend == ___)

abs(mean(boring_weekend == ___) - 0.05)
```

Comment: Based on both the full frequency table from Step 2 and the natural-20 rate here, does the die appear to be fair? Explain why.

Tip: A fair die should have counts that are roughly even overall and a natural-20 rate close to 0.05, but not perfectly exact.

13.5 Task 3

Scenario. A hospital is considering ordering new blankets for newborns. They want to know how many blankets they should order in each size. The hospital has data on newborn lengths, which are normally distributed with a mean of 20 inches and a standard deviation of 0.9 inches. The hospital wants to know how many babies are likely to be longer than 23 inches, which would require larger blankets.

13.5.1 Step 1

Baby-blanket test

Simulate 100 newborn lengths (inches) with mean 20.0, SD 0.9. Store as `blanketed_babies`. Use `rnorm()`.

```
#
blanketed_babies <- rnorm(___, mean = ___, sd = ___)
```

Comment: What do you expect the distribution of baby lengths to look like? Will the values be exactly equal? Have a normal distribution? Something else? Type your answer before working ahead. *Keep whatever you typed, even if you are wrong. That's the integrity of science!*

13.5.2 Step 2

Too long?

Using `any()`, check whether any babies in `blanketed_babies` are longer than 23 inches. Then count how many are over 23 inches. Do not store the result as a new object.

```
#
any(_________________)

sum(_________________)
```

Comment: Yes or No — based on this simulation, does the hospital appear to need longer blankets?

13.5.3 Step 3

Week-to-week risk

Repeat Steps 1 and 2 for 1000 simulated weeks. Store a vector named `weeks_too_long` that records, for each simulated week, **how many babies were longer than 23 inches**. Hint: use `replicate()`.

```
weeks_too_long <- replicate(___, sum(rnorm(___, mean = ___, sd = ___) > ___))
```

```r
#
table(weeks_too_long)
mean(weeks_too_long)
mean(weeks_too_long > 0)
max(weeks_too_long)
```

Comment: In these 1000 simulated weeks, how often do you see at least one baby longer than 23 inches? What does that suggest about whether the hospital should keep some larger blankets available?

13.6 Task 4

Reflect

Write a short paragraph reflecting on the utility of `replicate()` in the week-to-week risk simulation from Task 3, Step 3.

EXPLANATION: `"___"`

13.7 Save and Upload

1. You will be submitting **both** the `chapter15_practice.qmd` Document **and** the workspace file (`.RData`). Submit the rendered `chapter15_practice.html` file if you successfully rendered it and want to share it, but it is optional.

Review The workspace file (`.RData`) is all the objects in your environment that you created in this chapter.

Save your work and submit to Canvas.

Step 1: Sweep your Environment (start clean).
Click the **broom icon** in the Environment pane to clear objects.

Step 2: Run everything top-to-bottom.
Use **Run → Run All** to execute every chunk in order.

Step 3: Save and submit.

You can save the workspace (objects in your environment) by running the following command in a code chunk of the Quarto Document document:

```r
save.image("chapter15_practice.RData")
```

Or you can click the “Save workspace as” button in the Environment pane.